Wersjonowanie programów w Git

W Gicie do oznaczania wydań programu służą przede wszystkim **tagi** (`git tag`). Dzięki nim można oznaczyć konkretny commit jako wersję programu, np. `1.0.0`, `1.1.0`, `2.0.0` albo `28.6.163`.

Warto rozróżnić dwie rzeczy:

- **Commit** zapisuje konkretną zmianę w historii projektu.
- **Tag** oznacza konkretny punkt historii jako wersję programu.

1. Numerowanie wersji programu

Popularny schemat to Semantic Versioning, czyli `MAJOR.MINOR.PATCH`:

Wersja	Znaczenie
`1.0.0`	Pierwsze stabilne wydanie
`1.0.1`	Poprawka błędu
`1.1.0`	Nowa funkcja zgodna z dotychczasowym sposobem działania
`2.0.0`	Zmiana niezgodna ze starszymi wersjami

To konwencja, a nie wymóg Gita. Możesz przyjąć własny schemat, np. `28.6.163`, i samodzielnie określić znaczenie poszczególnych liczb. Ostatni człon może oznaczać numer kompilacji, rewizji lub wydania poprawkowego.

2. Oznaczenie wersji w Gicie

Załóżmy, że projekt znajduje się w repozytorium `moj-projekt`, a jego aktualny stan chcesz oznaczyć jako wersję `1.0.0`.

Najpierw sprawdź stan repozytorium:

```
git status
```

Jeśli są zmiany, które mają wejść do wydania, zapisz je w commicie:

```
git add .
git commit -m "Przygotowanie wersji 1.0.0"
```

Następnie utwórz tag:

```
git tag -a v1.0.0 -m "Wersja 1.0.0"
```

Tag `v1.0.0` wskazuje teraz commit, na którym stoisz. Opcja `-a` tworzy tag opisany (annotated), a `-m` ustawia jego opis.

Prefiks `v` jest popularną konwencją, ale nie jest obowiązkowy. Możesz używać również `1.0.0` bez litery `v`.

Wyświetlanie wersji

Lista tagów:

```
git tag
```

Informacje o konkretnym tagu:

```
git show v1.0.0
```

Zobaczysz opis tagu oraz commit, do którego się odnosi.

3. Tworzenie kolejnych wersji

Po naprawieniu błędu zapisz zmianę i oznacz nowe wydanie:

```
git add .
git commit -m "Naprawa błędu odczytu konfiguracji"
git tag -a v1.0.1 -m "Wersja 1.0.1"
```

Po dodaniu nowej funkcji:

```
git add .
git commit -m "Dodanie obsługi argumentów"
git tag -a v1.1.0 -m "Wersja 1.1.0"
```

Każdy tag wskazuje konkretny commit. Utworzenie nowego wydania nie zmienia wcześniej utworzonych tagów.

4. Wysłanie tagu na serwer

Utworzenie tagu lokalnie **nie wysyła go automatycznie** na GitHub, GitLab ani własny serwer.

Wyślij konkretny tag:

```
git push origin v1.0.0
```

Albo wszystkie lokalne tagi:

```
git push origin --tags
```

Po wysłaniu tagu będzie on dostępny w repozytorium zdalnym. Na platformach takich jak GitHub lub GitLab można następnie utworzyć wydanie (*Release*) i dołączyć np. plik wykonywalny, archiwum ze źródłami lub opis zmian.

5. Powrót do konkretnej wersji

Załóżmy, że istnieją tagi `v1.0.0`, `v1.0.1` i `v1.1.0`, a chcesz sprawdzić kod z wersji `v1.0.0`.

Wyświetl historię:

```
git log --oneline --decorate --all
```

Przejdź do oznaczonego commitu:

```
git switch --detach v1.0.0
```

Teraz oglądasz kod z wersji `v1.0.0`. Stan ten nazywa się *detached HEAD* — nie pracujesz na zwykłej gałęzi. Jest to wygodne do przeglądania lub testowania starszej wersji.

Jeśli chcesz dalej rozwijać starszą wersję, utwórz z niej gałąź:

```
git switch -c poprawki-1.0 v1.0.0
```

Możesz w ten sposób przygotować poprawkę dla linii `1.0.x`, nie zmieniając głównej linii rozwoju programu.

6. Numer wersji wewnątrz programu

Tag Git nie zmienia automatycznie numeru wersji wyświetlanego przez program. Jeśli aplikacja ma obsługiwać polecenie:

```
./moj-program --version
```

i wyświetlać np. `1.0.0`, trzeba osobno zapewnić jej dostęp do numeru wersji.

Jednym z prostych rozwiązań jest plik `VERSION`. Przykład dla wydania `1.0.0`:

```
echo "1.0.0" > VERSION
git add VERSION
git commit -m "Ustawienie wersji 1.0.0"
git tag -a v1.0.0 -m "Wersja 1.0.0"
git push origin main
git push origin v1.0.0
```

W tym przykładzie plik `VERSION` i tag zawierają ten sam numer. Najpierw zapisujesz plik w commicie, a dopiero potem oznaczasz ten commit tagiem.

Uwaga: wykonuj powyższe polecenia tylko wtedy, gdy tag `v1.0.0` jeszcze nie istnieje. Jeśli już istnieje, nie twórz go ponownie — upewnij się, że wskazuje właściwy commit. Jeśli główna gałąź nazywa się inaczej niż `main`, użyj właściwej nazwy.

W większych projektach numer wersji można automatycznie pobierać z tagu podczas kompilacji. Dzięki temu nie trzeba ręcznie aktualizować numeru w kilku miejscach.

7. Najważniejsze polecenia

Polecenie	Działanie
`git tag`	Wyświetla lokalne tagi
`git tag -a v1.0.0 -m "Wersja 1.0.0"`	Tworzy opisany tag
`git show v1.0.0`	Pokazuje informacje o tagu i wskazanym commicie
`git push origin v1.0.0`	Wysyła jeden tag na serwer
`git push origin --tags`	Wysyła wszystkie lokalne tagi
`git switch --detach v1.0.0`	Przechodzi do kodu oznaczonego daną wersją
`git switch -c poprawki-1.0 v1.0.0`	Tworzy gałąź od konkretnej wersji

Zalecany sposób pracy

- Używaj **commitów** do zapisywania zmian.
- Używaj **gałęzi** do niezależnego rozwijania funkcji i przygotowywania poprawek.
- Używaj **tagów** `v1.0.0`, `v1.0.1`, `v1.1.0` itd. do oznaczania wydań.
- Przed utworzeniem tagu sprawdź `git status` i upewnij się, że wszystkie zmiany przeznaczone do wydania zostały zapisane w commicie.
- Nie przesuwaj ani nie nadpisuj opublikowanych tagów bez wyraźnego powodu. Tag wydania powinien jednoznacznie wskazywać właściwy commit.